SAMPLE

COMP4422 Computer Graphics

Sample Mid-term Practice Script

Purpose: This practice script helps students become familiar with the mid-term assessment format, wording, and expected depth of answers. The questions are adapted from the course topics but use modified contexts and phrasing.

Total marks: 40 marks in this Sample

Part A: Multiple Choice (10 marks)

Instructions: Answer all questions. Select one best answer for each question. Each question carries 2 marks.

A1. In a standard graphics pipeline, which stage converts geometric primitives such as triangles into fragments that can later be coloured?

A. Vertex processing

B. Rasterization

C. Model transformation

D. Framebuffer output

A2. In GLSL ES 3.00 (used in WebGL2), which storage qualifier declares a global variable whose value remains constant across all vertex and fragment shader invocations during a single draw call?

A. uniform

B. in

C. out

D. local

A3. Why are Vertex Buffer Objects useful in real-time graphics applications?

A. They store the final image for display

B. They keep vertex data in GPU memory for efficient access

C. They automatically create texture images

D. They replace the need for shaders

A4. Which statement best describes a perspective projection?

A. Distant objects keep the same apparent size as nearby objects

B. Parallel lines always remain parallel on the screen

C. Distant objects generally appear smaller than nearby objects

SAMPLE
D. It is only used for engineering drawings

A5. A fragment shader is primarily responsible for:

A. Transforming object vertices into screen-related positions

B. Combining vertices into triangles

C. Computing properties such as the colour of individual fragments

D. Uploading geometry from CPU memory to GPU memory

Part B: True / False (10 marks)

Instructions: Write True or False for each statement. If the statement is True, no explanation is required. If the statement is False, briefly explain what is incorrect and provide the correct concept. Each question carries 2 marks: 1 mark for the correct choice and, for a false statement, 1 mark for a correct explanation.

C1. Fragment shaders execute once for every vertex in a rendered object. True / False

Explanation if False: _____

C2. Rasterization converts geometric primitives, such as triangles, into fragments for later processing. True / False

Explanation if False: _____

C3. A uniform variable can normally have a different value for every vertex within the same draw call. True / False

Explanation if False: _____

C4. In perspective projection, objects farther from the camera usually appear smaller, which provides a depth cue. True / False

Explanation if False: _____

C5. Keeping vertex data in a Vertex Buffer Object can improve performance because it reduces repeated data transfer between CPU memory and GPU memory. True / False

Explanation if False: _____

Part C: Short Questions (20 marks)

Instructions: Answer all questions. Use concise technical explanations. Diagrams are welcome where they help communicate your ideas.

B1. Graphics Pipeline Fundamentals (10 marks)

1. Put the following stages into a sensible order for transforming a 3D vertex to a screen position: Viewport transformation, Model transformation, Perspective division, Projection transformation, View transformation. (5 marks)

2. Name two primitive types that a graphics pipeline can process. (2 marks)

SAMPLE
3. Explain one performance advantage of storing mesh vertex data on the GPU rather than repeatedly sending it from the CPU for every frame. (3 marks)

B2. Projection and Shaders (10 marks)

A virtual museum application displays both an architectural floor plan and a realistic 3D walkthrough.

1. Which projection type would be more suitable for the floor plan: orthographic or perspective? Explain your choice using object size and parallel lines. (4 marks)

2. Which projection type would be more suitable for the 3D walkthrough? Explain your choice using depth perception. (2 marks)

3. Describe the different roles of a vertex shader and a fragment shader when rendering a textured 3D model. (4 marks)

SAMPLE

Appendix: Marking Scheme

This appendix is for self-checking after attempting the practice script.

Part A: Answers

- A1: B — Rasterization.
- A2: A — uniform.
- A3: B — Vertex data is retained in GPU memory for efficient access.
- A4: C — Objects farther away generally appear smaller.
- A5: C — Fragment shaders compute properties such as fragment colour.

Part B: Marking Scheme

For each question, award 1 mark for the correct True/False choice. For a False statement, award the second mark only when the explanation correctly identifies and corrects the error. For a True statement, award the full 2 marks for the correct choice; no explanation is required.

- B1: False. Fragment shaders operate on fragments/pixel candidates, not once per vertex. Vertex shaders execute per vertex.
- B2: True.
- B3: False. A uniform has one shared value during a draw call; it does not vary per vertex. Per-vertex data is supplied through vertex attributes and may be interpolated for fragments.
- B4: True.
- B5: True.

Part C: Marking Scheme

C1. Graphics Pipeline Fundamentals (10 marks)

- B1.1 (5 marks): Model transformation → View transformation → Projection transformation → Perspective division → Viewport transformation. Award 1 mark per correctly placed stage.
- B1.2 (2 marks): Any two valid primitive types, such as points, lines, or triangles. Award 1 mark each.
- B1.3 (3 marks): Award 1 mark for stating that data is stored in GPU memory/VRAM (for example, in a VBO); award up to 2 marks for explaining that this avoids or reduces repeated CPU-to-GPU transfer overhead and enables faster GPU access.

C2. Projection and Shaders (10 marks)

- B2.1 (4 marks): Orthographic projection. Award 2 marks for explaining that apparent object size does not vary with distance, and 2 marks for explaining that parallel lines remain parallel (no vanishing-point convergence).
- B2.2 (2 marks): Perspective projection. Award 2 marks for explaining that it provides depth/realism because distant objects appear smaller and parallel lines may converge.
- B2.3 (4 marks): Vertex shader (2 marks): operates per vertex, transforming positions and processing/passing attributes such as texture coordinates. Fragment shader (2 marks): operates per fragment and computes final colour, potentially using texture lookup, lighting, or other effects.

End of sample practice script.